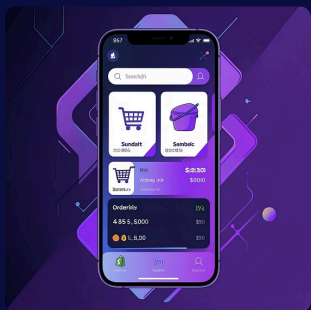


Taller Práctico — Análisis Crítico de Aplicaciones

Un taller sobre decisiones de arquitectura móvil usando dos casos reales — Shopify (React Native) y eBay (Flutter).



1 — Ingeniería inversa de la decisión



Shopify — Motivos pragmáticos

React Native permitió a Shopify reutilizar la experiencia conceptual y la lógica empresarial ya consolidada en React web. La decisión persigue coherencia en la lógica, velocidad de iteración y reducción de la curva de aprendizaje para desarrolladores front-end.



eBay — Motivo visual y sincronización

eBay buscó paridad funcional y fidelidad visual. Flutter ofrecía control completo sobre el renderizado y alto porcentaje de código compartido, reduciendo la ventana entre entregas en iOS y Android.

Instrucción para el equipo: desmontar las decisiones observando repositorios públicos, articulando trade-offs entre velocidad de entrega, cohesión lógica y control visual.

2 — Análisis de rendimiento y arquitectura (Shopify)

Prioridad técnica

Integración con periféricos (lectores, impresoras) exige latencia mínima.

Migración a Fabric

Reducir el Bridge clásico usando JSI para llamadas sincrónicas a APIs nativas.

Módulos nativos críticos

Implementar Native Modules para operaciones sensibles a tiempo real y mantener la UI a 60 FPS.

Recomendación técnica: encapsular la lógica de periféricos en módulos nativos con contratos bien definidos (promesas y callbacks optimizados) y reservar el hilo de JS para orquestación, no para bucles de I/O de baja latencia.

2 — Análisis de rendimiento y arquitectura (eBay)



Render propio (Skia / Impeller)

Flutter dibuja su propia UI, reduciendo costes de traducción y permitiendo micro-interacciones consistentes incluso en dispositivos de gama media.



Gestión intensiva de imágenes

Cachés de imágenes, decodificación en background y placeholders para mantener 60 FPS durante scroll y animaciones.



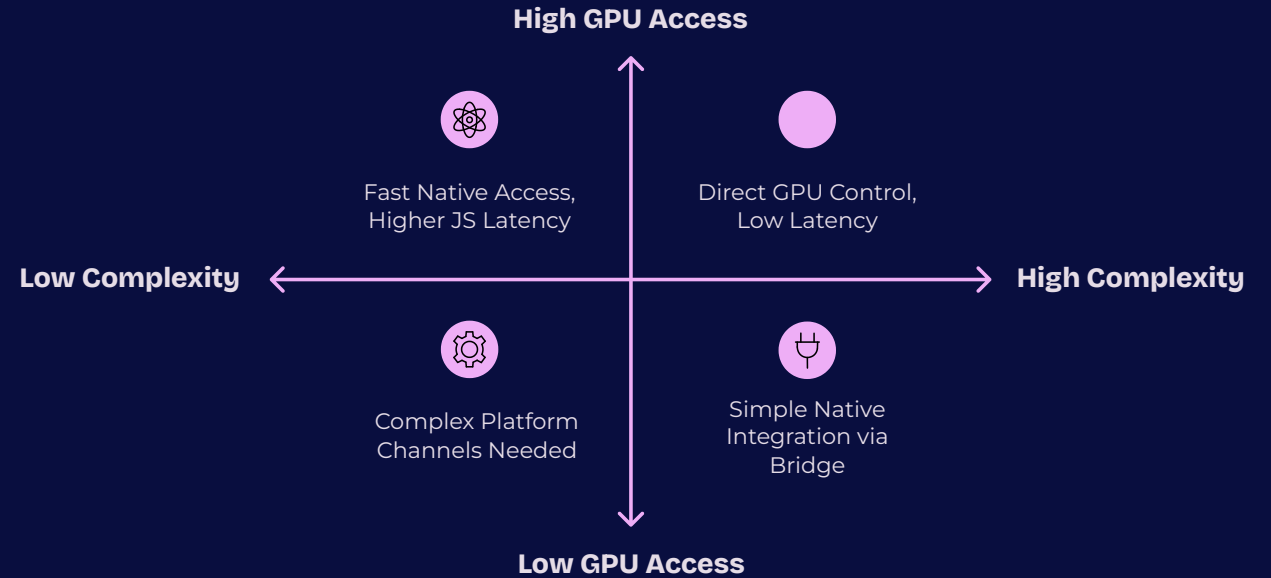
Micro-interacciones y prioridad de frames

Optimizar pipelines de render y offload de trabajo pesado a isolates o capas nativas para evitar jank.

Acción sugerida: instrumentar perfiles reales en dispositivos objetivo, medir frames lost y tiempos de rasterización; priorizar optimizaciones en el pipeline de imágenes y reducir trabajo en el hilo principal del UI.

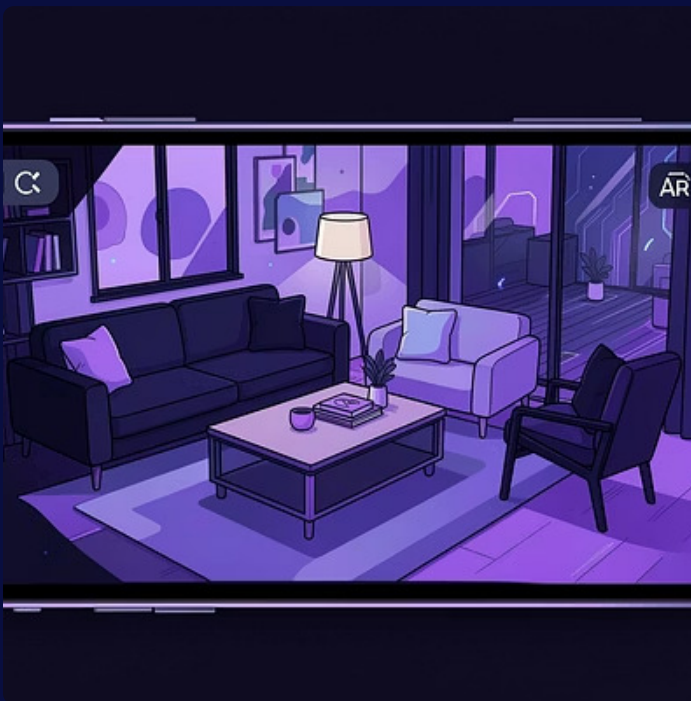


3 — El desafío de las nuevas pantallas



Interpretación: React Native actúa como host, facilita acceso nativo rápido pero su dependencia del hilo JS puede introducir latencia. Flutter controla el renderizado pero necesita bridges para APIs gráficas de bajo nivel; ambos enfoques requieren módulos nativos para experiencias AR de baja latencia.

3 — AR y rendimiento: puntos concretos



1. Latencia de comunicación

Minimizar viajes entre capas (JS $\leftrightarrow$ nativo). Para AR, mover lógica crítica a nativo es preferible.



2. Acceso y control GPU

Shaders y render en tiempo real requieren acceso cercano al hardware; la abstracción añade complejidad.



3. Integración con sensores

Fusión de sensores (IMU, cámara) debe implementarse preferentemente en nativo para procesamiento a baja latencia.

4 — Veredicto crítico

1 Elección final propuesta: Nativo

Para aplicaciones que requieren mapas 3D, notificaciones de alta frecuencia y soporte robusto para pantallas plegables, la opción nativa ofrece control más directo del hardware, mejores garantías de rendimiento y acceso temprano a APIs del sistema.

3 Razón — Notificaciones y servicios en segundo plano

Los subsistemas nativos permiten mayor resiliencia frente a políticas de terminación del SO y a modelos de hilos multihilo complejos.

2 Razón — Mapas 3D

Renderizado intensivo y gestión de memoria se controlan mejor con acceso directo a GPU y lifecycle nativo.

4 Razón — Pantallas plegables

APIs específicas (WindowManager, continuity APIs) preceden a los bindings multiplataforma; la adopción nativa es más rápida y estable.

¿Qué es un aplicativo nativo y cómo se clasifica?

Un aplicativo nativo es un programa diseñado específicamente para un sistema operativo móvil (iOS o Android), utilizando sus lenguajes y herramientas propias. Esto asegura una integración perfecta con el hardware y software del dispositivo, optimizando rendimiento y experiencia de usuario.

1 Por Plataforma de Destino

- **iOS Nativo:** Desarrollado con Swift u Objective-C, usando Xcode y SDKs de Apple (UIKit, SwiftUI, ARKit).
- **Android Nativo:** Desarrollado con Kotlin o Java, usando Android Studio y SDKs de Google (Jetpack Compose, ARCore).

2 Por Tipo de Dispositivo

- **Móvil:** Para smartphones, aprovechando sus capacidades completas.
- **Wearables:** Como Apple Watch o Wear OS, requieren frameworks ligeros por sus recursos limitados.
- **Tablets y Plegables:** Exigen diseños adaptativos para cambios de pantalla en tiempo real.

3 Por Framework de UI Nativo

- **Ecosistema Apple:** UIKit / SwiftUI.
- **Ecosistema Android:** XML Layouts / Jetpack Compose.

¡Gracias por su atención!